

Code Hardener

ATTESTATION CERTIFICATE



bulletproof-process-knowledge-mcp

Scan e0f3fea6 | Branch: main | Jul 24, 2026

Attestation ID

e54156ff-3cc0-4da8-85ba-...

Type

in-toto

Subject

bulletproof-process-knowledge-mcp

Created

Jul 24, 2026, 3:28 PM

Subject Digest

sha256: 41eaa5a3a944adb440a2c461acda0b83d6611ab195e34eb6075d574a...

Signature Status

CRYPTOGRAPHICALLY SIGNED

Algorithm: Ed25519 (Local Signing Key)

Signature

3tRTVYkSKETmyvkbRgNuRfNx6qTIBsNa0J2FdnxmPSvvFX/E6DYavrR5xU7swgcv
bcKvKVryJ4SMK1SAhTs2Bw==

This attestation certifies the security scan was performed and results were recorded with cryptographic integrity.

Scan Report — bulletproof-process-knowledge-mcp

Branch: main | Status: completed | Jul 24, 2026

992 / 1000 Score — Quality: excellent

Scan Details

Profile: standard
Duration: 14s
Started: Jul 24, 2026, 3:28 PM
Completed: Jul 24, 2026, 3:28 PM
Files Analyzed: 8
Scanners Run: 11 of 12
Open Findings: 4
Resolved / Suppressed:0
Total Detected: 4
Codebase: Markdown: 2 files | Python: 1 files | Config: 1 files | Docker: 1 files

0 Critical 0 High 0 Medium 4 Low 0 Info

Scanner Results

12 tools executed — 9 passed | 2 with open findings | 1 not applicable | 0 errors

Scanner	Category	Outcome	Findings	Duration	Files Scanned
trivy	SCA	FAIL	2	2s	2
gitleaks	Secrets	PASS	0	0s	8
opengrep	SAST	PASS	0	5s	1
checkov	IaC	PASS	0	2s	2
grype	SCA	PASS	0	1s	8
syft	SBOM	FAIL	2	2s	2
package-validator	AI Code Quality	PASS	0	0s	1

oxlint	Code Quality	N/A	0	0s	1
ruff	Code Quality	PASS	0	0s	1
actionlint	CI/CD Security	PASS	0	0s	1
jscpd	Code Quality	PASS	0	0s	1
typos	Code Quality	PASS	0	0s	8

Scanner Evidence & Audit Trail

trivy (SCA)

FINDINGS

Scan Scope: Scanned 2 target(s): dockerfile

Files Analyzed: 2

Rules Checked: 4

Checks Performed:

- Vulnerability scanning
- Misconfiguration detection
- Secret detection
- License compliance

Standards: CVE/NVD, OWASP Top 10 (A06), CIS Benchmarks, NIST SP 800-53

Methodology:

Scans filesystem for package manifests and lock files, matches installed packages against vulnerability databases (NVD, GitHub Advisory, vendor-specific). Also checks for misconfigurations and secrets using built-in rules.

Targets Analyzed:

Dockerfile, Loose File License(s)

gitleaks (Secrets)

PASS

Scan Scope: Working directory scanned for 150+ secret patterns with entropy analysis

Configuration: Custom gitleaks.toml

Files Analyzed: 8

Rules Checked: 150

Checks Performed:

- AWS credential detection
- API key detection
- Private key detection
- Generic secret patterns
- High-entropy string analysis
- Token detection

Standards: CWE-798, CWE-259, OWASP Top 10 (A07)

Methodology:

Regex-based pattern matching against 150+ secret patterns with entropy analysis. Scans current file contents and optionally git commit history. Applies allowlists and custom rules to reduce false positives. Secrets are redacted in output.

opengrep (SAST)

PASS

Scan Scope: Multi-language SAST with auto, security-audit, owasp-top-ten, cwe-top-25, r2c-security-audit, and secrets rulesets

Configuration: Custom semgrep-rules.yml + default rulesets

Files Analyzed: 1

Checks Performed:

- OWASP Top 10 patterns
- Security audit rules

- Secrets detection
- Injection patterns
- XSS patterns
- Insecure crypto usage

Standards: OWASP Top 10, CWE Top 25, SANS Top 25

Methodology:

Pattern-based static analysis using Semgrep rule engine with auto, security-audit, owasp-top-ten, and secrets rulesets. Matches AST patterns against known vulnerability signatures.

checkov (IaC)

PASS

Scan Scope: IaC security analysis — 0 files, 0 checks (0 passed, 0 failed), frameworks: auto-detected

Configuration: Custom .checkov.yaml

Files Analyzed: 2

Checks Performed:

- Terraform misconfigurations
- CloudFormation template issues
- Kubernetes manifest hardening
- Dockerfile best practices
- Helm chart security
- GitHub Actions security
- GitLab CI security
- Secrets in IaC files

Standards: CIS Benchmarks, NIST SP 800-53, SOC 2, PCI-DSS, HIPAA

Methodology:

Graph-based static analysis of infrastructure-as-code files. Parses IaC definitions, builds resource graphs, and checks 1000+ built-in policies covering security, networking, encryption, and compliance.

grype (SCA)

PASS

Scan Scope: SCA analysis: 0 unique CVEs across 0 vulnerable packages (filesystem)

Configuration: Grype with --by-cve deduplication and --add-cpes-if-none

Files Analyzed: 8

Checks Performed:

- CVE vulnerability matching
- Dependency version analysis
- CPE-based matching
- Advisory cross-referencing
- Fix availability detection

Standards: CVE/NVD, OWASP Top 10 (A06), GHSA

Methodology:

Generates SBOM using Syft, then matches all identified packages against vulnerability databases (NVD, GitHub Advisories, OS vendor databases). Reports CVEs with severity, fix versions, and references.

syft (SBOM)

FINDINGS

Scan Scope: SBOM of 2 packages across 0 language(s): N/A

Configuration: Syft with JSON + CycloneDX output for SBOM standards compliance

Files Analyzed: 2

Rules Checked: 4

Checks Performed:

- SBOM generation (Syft JSON + CycloneDX)
- License compliance analysis
- Copyleft license detection (AGPL, GPL, MPL)
- Weak copyleft classification (LGPL)
- Unknown license flagging

- Package URL (PURL) generation
- CPE generation

Standards: NTIA SBOM Minimum Elements, CycloneDX, SPDX, Executive Order 14028

Methodology:

Catalogs software packages by parsing package manifests, lock files, and installed package databases. Generates standardized SBOM in CycloneDX or SPDX format for supply chain visibility.

package-validator (AI Code Quality)

PASS

Scan Scope: Validated 6 packages across 1 dependency file(s)

Configuration: Concurrency: 10, Timeout: 5000ms per request

Files Analyzed: 1

Rules Checked: 1

Checks Performed:

- npm registry existence validation
- PyPI registry existence validation
- Go module proxy existence validation
- Slopsquatting risk detection
- Hallucinated dependency identification

Standards: CWE-829, OWASP A08:2021, SLSA Supply Chain Security

Methodology:

Parses dependency manifests and validates each package exists in its respective registry (npm, PyPI, Go proxy) via HTTP HEAD requests. Non-existent packages are flagged as hallucinated — a common AI code generation artifact that creates supply chain attack vectors.

oxlint (Code Quality)

N/A

Scan Scope: JavaScript (.js, .jsx) and TypeScript (.ts, .tsx) source files

Files Analyzed: 1

Checks Performed:

- Code correctness checks
- Suspicious code detection
- Performance anti-patterns
- Import/export validation

Standards: CWE, ESLint Core Rules

Methodology:

Rust-based linter that performs AST analysis of JS/TS files at high speed. Detects correctness issues, suspicious patterns, and performance anti-patterns.

ruff (Code Quality)

PASS

Scan Scope: Python linting and security analysis via Ruff

Configuration: Default Ruff rules

Files Analyzed: 1

Checks Performed:

- Pyflakes error detection
- pycodestyle enforcement
- flake8-bandit security checks
- isort import ordering
- pep8-naming conventions
- pyupgrade modernization
- flake8-bugbear bug detection
- flake8-comprehensions optimization

Standards: PEP 8, PEP 257, CWE, OWASP Top 10

Methodology:

Rust-based Python linter implementing 800+ rules from flake8, pylint, isort, and others. 10-100x faster than traditional Python

linters.

actionlint (CI/CD Security)

PASS

Scan Scope: .github/workflows/*.yml files

Files Analyzed: 1

Checks Performed:

- Workflow syntax validation
- Action version pinning checks
- Shell script validation
- Expression type checking
- Permissions analysis

Standards: GitHub Actions Security Best Practices, CWE-78

Methodology:

Static analysis of GitHub Actions workflow files. Validates syntax, checks for unpinned action versions, analyzes shell scripts for errors, and verifies expression types.

jscpd (Code Quality)

PASS

Scan Scope: All source files across supported languages

Files Analyzed: 1

Checks Performed:

- Cross-file duplicate detection
- Code clone identification
- Duplication percentage calculation

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Token-based analysis to detect copy-pasted code blocks across files and languages. Reports duplication percentage and identifies specific cloned regions.

typos (Code Quality)

PASS

Scan Scope: All text files in the project

Files Analyzed: 8

Checks Performed:

- Identifier spell checking
- String literal spell checking
- Comment spell checking
- Filename spell checking

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Fast, low-false-positive spell checker designed for source code. Checks identifiers, strings, and comments against a curated dictionary.

Open Findings (4)

syft

SBOM | 2 findings | Software bill of materials generator for supply chain visibility

Severity	Title	File	CWE / Rule
LOW	Unknown License: actions/setup-python@a26af69be951a21... Package actions/setup-python has no license information. This may indicate a compliance risk.	/.github/workflows/ci.yml	SBOM-LICENSE-UNKNOWN

Package: actions/setup-python
Version: a26af69be951a213d495a4c3e4e4022e16d87065
Type: github-action

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

LOW Unknown License: actions/checkout@11d5960a326750d5838... /github/workflows/ci.yml SBOM-LICENSE-UNKNOWN
Package actions/checkout has no license information. This may indicate a compliance risk.

Package: actions/checkout
Version: 11d5960a326750d5838078e36cf38b85af677262
Type: github-action

Fix: Verify the license for this package before using in production.
OWASP: A08:2021-Software and Data Integrity Failures

trivy

SCA | 2 findings | Container image and filesystem vulnerability scanner

Severity	Title	File	CWE / Rule
LOW	License Compliance: Apache-2.0 in Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy.	LICENSE	LICENSE-Apache-2.0

Package:
License: Apache-2.0
Category: notice

Fix: Review this finding and apply the appropriate fix based on the description: Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy.
OWASP: A06:2021-Vulnerable and Outdated Components

LOW No HEALTHCHECK defined Dockerfile DS-0026
You should add HEALTHCHECK instruction in your docker container images to perform the health check on running containers.
Fix: Add HEALTHCHECK instruction in Dockerfile
OWASP: A05:2021-Security Misconfiguration

Codebase Coverage (8 files analyzed)

This scan analyzed 8 files using 11 security tools across 8 testing categories. Each scanner targets files relevant to its analysis domain (e.g., SAST scanners analyze source code, SCA scanners analyze dependency manifests).

Files by Language:

Markdown: 2 files Python: 1 files
Config: 1 files Docker: 1 files

Files by Extension:

.other: 2 .yaml: 1 .dockerfile: 1
.gitignore: 1 .md: 1 .py: 1
.txt: 1

Per-Scanner File Coverage:

Scanner	Category	Files Analyzed	Scope
trivy	SCA	2	Scanned 2 target(s): dockerfile
gitleaks	Secrets	8	Working directory scanned for 150+ secret p...
opengrep	SAST	1	Multi-language SAST with auto, security-aud...
checkov	IaC	2	IaC security analysis — 0 files, 0 checks (...)
grype	SCA	8	SCA analysis: 0 unique CVEs across 0 vulner...
syft	SBOM	2	SBOM of 2 packages across 0 language(s): N/A
package-validator	AI Code Quality	1	Validated 6 packages across 1 dependency fi...
oxlint	Code Quality	1	JavaScript (.js, .jsx) and TypeScript (.ts,...)
ruff	Code Quality	1	Python linting and security analysis via Ruff
actionlint	CI/CD Security	1	.github/workflows/*.yaml files
jscpd	Code Quality	1	All source files across supported languages
typos	Code Quality	8	All text files in the project

Files with Findings (3):

File Path	Findings	Scanners
/.github/workflows/ci.yml	2	syft
Dockerfile	1	trivy
LICENSE	1	trivy

Software Bill of Materials (2 packages)

Package types: github-action: 2

Top licenses: Unknown (2)

Package	Version	Type	Language	License
actions/checkout	11d5960a326750	github-actio	-	Unknown
actions/setup-python	a26af69be951a2	github-actio	-	Unknown